CineRoll — Reparto de responsabilidades, Fase 2

Plataforma de Cine, Series y Anime · Actualizado el 8 de noviembre de 2026

1. Dónde estamos

La Fase 1 (base del proyecto) está cerrada. Revisando `main`, el equipo incluso adelantó trabajo que el roadmap tenía para fases posteriores. Este es el estado real:

Área	Estado en main
Base del proyecto	Django + DRF + JWT + CORS, settings divididas, PostgreSQL (o SQLite con `USE_SQLITE`), `/api/health/`. Listo.
Catálogo (media)	Modelos `Media`, `Genero`, `Persona`, `Credito`; `seed_data` con 37 fichas; API de solo lectura con búsqueda, filtros y paginación; Django Admin. Listo.
Cartelera (cines)	Modelos `Cine`, `Funcion`, `TituloPendiente`; scraper de Cinemark; normalización y matching de títulos; comando `scrape_cartelera`; endpoints `/api/cines/` y `/api/media/<id>/funciones/`; admin para revisar títulos pendientes. Listo para 1 cine.
Usuarios y reseñas	Usuario propio, registro, login, refresh, perfil, escala de calificación, permisos reutilizables, reseñas con promedio y moderación en el admin. Listo.
Frontend	Solo la pantalla de login/registro. El registro está en modo demostración y el login usa credenciales de prueba escritas en el código.

Esto significa que la Fase 2 del roadmap (API del catálogo, admin y permisos) está cubierta en su mayor parte. Lo que queda son los cabos sueltos de esa fase, más preparar el terreno para la Fase 3 (automatización) y la Fase 4 (social).

2. Cuántas fases son y el calendario

El roadmap tiene **8 fases (de la 0 a la 7)** en un semestre de 16 semanas. Las fases 0 y 1 ya terminaron; con esta, **quedan 6 por recorrer (2, 3, 4, 5, 6 y 7)**.

Las fechas son una propuesta mía, asumiendo que la semana 1 empezó el 5 de octubre y que la Fase 2 va del 9 al 22 de noviembre. Hay que confirmarlas contra el calendario académico real.

Fase	Objetivo	Fechas aprox.
0 — Investigación y diseño	Alcance, modelos, arquitectura	Terminada
1 — Base del proyecto	Esqueleto del backend funcionando	Terminada

Fase	Objetivo	Fechas aprox.
2 — API del catálogo	Cerrar API, admin y permisos	9 – 22 nov
3 — Scraping y cartelera real	Más cines, automatización con Celery	23 nov – 13 dic
4 — Capa social	Listas, seguidores, sugerencias de edición	14 – 27 dic
5 — Frontend	Interfaz completa que consume la API	28 dic – 17 ene
6 — Integración y pruebas	Pruebas end-to-end y rendimiento	18 – 24 ene
7 — Documentación y sustentación	Demo y presentación final	25 – 31 ene

3. Cuándo se trabaja el frontend

- **Trabajo formal de frontend: Fase 5**, del 28 de diciembre al 17 de enero (3 semanas). Ahí se construyen el home con cartelera y próximos estrenos, la ficha de cada título, el perfil y las reseñas.
- **Wireframes e identidad visual: desde la segunda semana de la Fase 2** (a partir del 16 de noviembre), a cargo de la Persona 4. Como el backend del MVP ya está estable, tiene sentido empezar a diseñar ahora para que en la Fase 5 se construya sobre algo ya decidido.
- **Durante la Fase 2 solo se toca el frontend actual** para dejarlo limpio: conectar el registro real y quitar lo que es de demostración. No se construyen pantallas nuevas todavía.

4. Objetivo de la Fase 2

Cerrar la API del catálogo y la cartelera para que el frontend tenga todo lo que necesita, y dejar listo lo que la Fase 3 va a usar (Celery, segundo cine, limpieza de cartelera vencida).

Reglas de trabajo: se mantienen las mismas del reparto anterior (una rama `feature/<area>-<descripcion>` por funcionalidad, PR obligatorio con al menos una revisión, commits pequeños, secretos solo en `.env`, migraciones solo de las apps propias y sincronización semanal de 15 a 20 minutos).

5. Persona 1 — Líder técnico e integración

Sigue siendo quien destraba al resto y mantiene `main` funcionando.

1. **Integración continua (primero, en la semana 1).** Configurar GitHub Actions para que cada PR corra `python manage.py test` contra PostgreSQL y no se pueda hacer merge si falla. Hasta ahora las pruebas solo se han corrido en SQLite, así que esta será la primera vez que corran en PostgreSQL: si algo se comporta distinto (por ejemplo, las migraciones de `usuarios`), conviene descubrirlo al inicio de la fase.
2. **Revisión y merge** de todos los PRs de la fase, resolviendo conflictos de migraciones entre apps.
3. **Settings de producción.** Crear `config/settings/production.py` (DEBUG apagado, `ALLOWED_HOSTS` y orígenes CORS desde `.env`). Hoy `wsgi.py`, `asgi.py` y `manage.py` apuntan siempre a `local`.

1. **Preparar Celery y Redis** (segunda semana): agregar Redis al `docker-compose.yml`, instalar Celery y crear `config/celery.py`, con una tarea de prueba. Así la Persona 3 puede automatizar los scrapers apenas empiece la Fase 3.
2. **Limpiar las dependencias del frontend.** El `package-lock.json` incluye `express`, `cors` y `jsonwebtoken`, que no están en `package.json`, y `axios` está declarado pero no se usa. Regenerar el lock para que `npm ci` funcione igual para todos.
3. **Decidir el stack del frontend** junto con la Persona 4: seguir con React + Vite (lo que ya existe) o pasar a Next.js, como permite el roadmap. Debe quedar decidido antes del 22 de noviembre.
4. **Formato de errores de la API (primera tarea de la semana 1).** Hoy el login responde `{"error": ...}` y el resto usa el formato estándar de DRF. Se usará un solo formato para toda la API, con un `EXCEPTION_HANDLER` propio de DRF en `config/exceptions.py`, registrado en `REST_FRAMEWORK` de `base.py`. Todas las respuestas de error tendrán la forma `{"error": {"codigo": "...", "mensaje": "...", "campos": {...}}}`, donde `campos` solo aparece en errores de validación. El login deja de armar su propia respuesta de error y lanza las excepciones de DRF. Va primero para que todos los endpoints nuevos de la fase nazcan con este formato.

Terminado cuando: cada PR corre las pruebas automáticamente, existe la configuración de producción, Celery arranca con una tarea de prueba y `npm ci` funciona sin errores.

6. Persona 2 — Catálogo (`media`)

1. **Cerrar el CRUD de la API.** El roadmap pide endpoints CRUD para Media, Género y Persona, pero hoy son de solo lectura (un POST devuelve 405). Agregar escritura solo para administradores, reutilizando `EsAdministradorOLecturaPublica`, y actualizar la prueba que hoy espera el 405.
2. **Endpoint de personas.** Crear `/api/personas/` (lista y detalle, con escritura para admin) para que el frontend pueda mostrar la ficha de un actor o director.
3. **Más opciones de listado.** Agregar ordenamiento (`?ordering=` por fecha de estreno, título o puntuación) y el filtro por cine (`?cine=<id>`), que el roadmap incluye en el catálogo. La Persona 2 es la única responsable del filtro por cine: lo implementa en `MediaListView` filtrando por `funciones__cine`, y la Persona 3 solo lo revisa en el PR. Para ordenar por puntuación se usa el campo `promedio_puntuacion`, cuyo nombre mantiene la Persona 4.
4. **Enriquecer `seed_data`.** Las fichas no tienen sinopsis, póster ni tráiler. Completarlos y agregar al menos 5 títulos como `proximo_estreno` con fecha futura, para que el home del frontend tenga datos reales de próximos estrenos. El comando debe seguir siendo idempotente.
5. **Pruebas** de los nuevos endpoints, del ordenamiento y de los permisos de escritura.

Terminado cuando: un administrador puede crear, editar y borrar Media, Género y Persona por la API, y un visitante solo puede leer. Con la base vacía, migraciones y `seed_data` dejan un catálogo con pósters y próximos estrenos.

7. Persona 3 — Cartelera y scraping (`cines`)

1. **Terminar el reconocimiento** de Cinépolis, Royal Films y Cineland desde un navegador, registrando en la pestaña Network la URL, los parámetros y la respuesta que traen las funciones. Es lo que quedó pendiente en `RECONOCIMIENTO_CARTELERA.md`. Cinépolis es el primer

candidato porque su selector de ciudad incluye Valledupar, mientras que la cartelera de Cinemark en Valledupar estaba vacía. El resultado define qué cine se agrega como segundo scraper en la Fase 3.

2. **Próximos estrenos desde Cinemark.** El scraper ignora hoy la sección `UpcomingReleases`. Leerla para tener fechas de estreno reales, acordando con la Persona 2 cómo se refleja en `Media`.
3. **Quitar cartelera vencida.** El comando solo marca títulos como `en_cartelera`; nunca los devuelve a `fuera_cartelera`, y las funciones pasadas quedan en la base. Definir y aplicar la regla (por ejemplo, un título sin funciones futuras vuelve a fuera de cartelera) acordándola con la Persona 2. La limpieza corre al final de cada ejecución de `scrape_cartelera`, sin esperar a Celery; en la Fase 3 la tarea programada solo llamará a ese mismo servicio.
4. **Mover la lógica del comando a un servicio.** Pasar el flujo de `scrape_cartelera` a una función en `services.py` que se pueda llamar desde un comando y desde una tarea de Celery. Esto prepara la Fase 3.
5. **Endpoints para el home.** Agregar `/api/cines/<id>/funciones/` (cartelera de un cine) y un listado de funciones filtrable por fecha y ciudad. Además, escritura de `Cine` solo para administradores, como pide el roadmap.
6. **Revisar el filtro `?cine=`** que implementa la Persona 2 en el catálogo, para confirmar que consulta bien la relación con `Funcion`.

Terminado cuando: el reconocimiento de los 3 cines restantes está completo y el cine del segundo scraper (Fase 3) está elegido, la cartelera vencida se limpia en cada ejecución del scraper, y el frontend puede pedir "qué hay hoy" y "qué hay en este cine" con una sola llamada.

8. Persona 4 — Usuarios, reseñas y preparación del frontend

1. **Cerrar la sesión de forma segura.** Hoy el refresh token rota pero el anterior sigue siendo válido (`BLACKLIST_AFTER_ROTATION` está en `False`). Activar la lista negra de tokens y crear `/api/auth/logout/`. Agregar también el cambio de contraseña desde el perfil. Para la lista negra hay que agregar `rest_framework_simplejwt.token_blacklist` a `INSTALLED_APPS`; esa app trae sus propias migraciones, así que se avisa a la Persona 1 antes de subirla.
2. **Endpoints que necesita el perfil del frontend:** la reseña propia de un título (para saber si ya reseñó y editarla) y "mis reseñas" en el perfil. Dejar listo un perfil público con las reseñas de un usuario, que además sirve de base para los seguidores de la Fase 4.
3. **Nombre de los campos de puntuación.** La Persona 4 es la única responsable de los nombres `promedio_puntuacion` y `total_votos` que agrega `social.services.anotar_puntuacion` al catálogo. La Persona 2 los usa tal cual para ordenar; si alguna vez cambian, la Persona 4 avisa antes.
4. **Diseñar (sin implementar) los modelos de la Fase 4:** `Lista` (favoritas, por ver), `Seguimiento` y `SugerenciaEdicion`. Presentar la propuesta de campos y relaciones al grupo para aprobarla antes de empezar a programar.
5. **Limpiar el frontend actual:** conectar el registro al endpoint real `/api/auth/registro/`, quitar el banner con credenciales de demo y los mensajes de prueba, y usar las funciones de `services/api.js` (incluido el refresh del token) en lugar del `fetch` duplicado dentro de `App.jsx`. Hacer que "Recordarme" tenga efecto o quitarlo.

6. **Wireframes e identidad visual** (desde el 16 de noviembre): home con cartelera y próximos estrenos, ficha de título, perfil y reseñas. Se presentan al grupo antes de terminar la fase.
7. **Pruebas** de logout, cambio de contraseña y los nuevos endpoints.

Terminado cuando: una persona puede registrarse desde la pantalla, iniciar sesión, cerrar sesión de forma que su token deje de servir, y los wireframes están aprobados por el grupo.

9. Puntos de integración de la Fase 2

Punto	Quiénes	Qué acordar	Cuándo
Cartelera vencida y estado de los títulos	Personas 2 y 3	Cuándo un título pasa a `en_cartelera`, `fuera_cartelera` o `proximo_estreno`. La limpieza corre al final de cada `scrape_cartelera`.	Semana 1
Filtro por cine en el catálogo	Persona 2	Implementa `?cine=<id>` en el catálogo filtrando por `funciones__cine`. La Persona 3 solo revisa el PR.	Semana 1
Escritura por la API (CRUD)	Personas 2, 3 y 4	Todos usan los permisos ya publicados, sin escribir los suyos	Semana 1
Formato de errores	Persona 1 (todos lo usan)	Un `EXCEPTION_HANDLER` de DRF con la forma `{"error": {"codigo", "mensaje", "campos"}}`, también en el login.	Semana 1, antes que todo lo demás
Puntuación en el catálogo	Persona 4	Mantiene los nombres `promedio_puntuacion` y `total_votos`, que la Persona 2 usa para ordenar.	Semana 1
Stack del frontend	Personas 1 y 4	React + Vite o Next.js	Antes del 22 nov
Celery y Redis	Personas 1 y 3	Cómo se define y se llama una tarea	Semana 2

10. Criterio de cierre de la Fase 2

- Todo el CRUD de catálogo y cines funciona por la API con permisos correctos.
- El cine del segundo scraper (Fase 3) está elegido y la cartelera vencida se limpia en cada ejecución del scraper.
- Toda la API responde los errores con un solo formato.
- Cada PR corre las pruebas en GitHub automáticamente.
- Celery y Redis están listos para empezar la Fase 3.
- El login y el registro del frontend usan la API real, sin datos de demostración.
- Los wireframes y el stack del frontend están decididos.

11. Documentación

La documentación en formato `.md` (README, guías, informes de reconocimiento y demás textos de apoyo) **no forma parte de las responsabilidades de esta fase**. Esos documentos se compartirán con el grupo por WhatsApp.